

Exam 16-01-2025 - Selected Solutions

Problem 1: Trucking Company

(a) Greedy Algorithm and Complexity

```
Algorithm GreedyTrucks(n, w, W):  
    n_trucks = 1  
    curr_W = 0  
  
    For i = 1 to n:  
        if curr_W + w[i] <= W:  
            curr_W = curr_W + w[i]  
        else:  
            n_trucks = n_trucks + 1  
            curr_W = w[i]  
  
    Return n_trucks
```

- **Time Complexity:** $O(n)$, poiché l'algoritmo scorre l'array dei pacchi una sola volta con un unico ciclo `For` [cite: 6].
- **Space Complexity:** $O(1)$ di memoria aggiuntiva, utilizzando solo variabili di supporto costanti.

(b) Ottimalità dell'algoritmo Greedy Il greedy inserisce un nuovo camion solo quando è strettamente costretto a farlo, ovvero quando aggiungere il pacco corrente farebbe superare la capacità massima W del camion in corso [cite: 6]. Nessun'altra soluzione valida (incluso l'ottimo OPT) può inserire più pacchi nello stesso camion senza violare il limite di peso W o l'ordine di arrivo sequenziale [cite: 6]. Per il principio del "Greedy Stays Ahead", il carico accumulato dal greedy a ogni passo è massimizzato, garantendo che il numero di camion utilizzato sia minimo.

Problem 2: Maximum Contiguous Subarray Sum

(a) Recursive Formula Sia $OPT[i]$ la somma massima del sottoarray contiguo che termina esattamente alla posizione i . La scelta per l'elemento $A[i]$ è se estendere il sottoarray precedente o iniziarne uno nuovo partendo da i :

$$OPT[i] = \max(A[i], OPT[i - 1] + A[i])$$

(b) Efficient Implementation and Complexity

```

Algorithm MaxSubarray(A, n):
    max_so_far = A[1]
    curr_max = A[1]

    For i = 2 to n:
        curr_max = max(A[i], curr_max + A[i])
        max_so_far = max(max_so_far, curr_max)

    Return max_so_far

```

- **Time Complexity:** $O(n)$, poiché l'array viene scorso una sola volta.
- **Space Complexity:** $O(1)$ di memoria ausiliaria, memorizzando solo variabili di stato costanti.

(e) Recupero del Subarray Effettivo Per ricostruire il sottoarray reale, è sufficiente tracciare gli indici di inizio e fine ogni volta che `curr_max` viene aggiornato o fatto ripartire da `i`, memorizzando gli indici globali in cui `max_so_far` raggiunge il suo picco massimo.

Problem 3: Independent Set is NP-Complete

1. Definizione di Riduzione Polinomiale: Una riduzione polinomiale da un problema di decisione A a un problema B è una funzione calcolabile in tempo polinomiale f tale che, per ogni istanza x , $x \in A$ se e solo se $f(x) \in B$.

2. NP-Hardness (Riduzione da 3-SAT): * **Costruzione del Gadget:** Data una formula 3-SAT con m clausole, creiamo un vertice per ciascuno dei tre letterali di ogni clausola, ottenendo $3m$ nodi raggruppati in m triangoli (uno per clausola). Aggiungiamo inoltre "archi di coerenza" tra ogni variabile x_i e la sua negazione $\bar{x}_i$ per impedire assegnazioni contraddittorie. * **Dimostrazione ($k = m$):** Poniamo la richiesta di trovare un Independent Set di dimensione almeno $k = m$ (un nodo per ogni clausola). Se la formula 3-SAT è soddisfacibile, l'assegnazione di verità permette di selezionare un letterale vero per ciascuna clausola nei triangoli senza violare gli archi di coerenza, ottenendo un Independent Set di dimensione m . Viceversa, se esiste un Independent Set di dimensione m , poiché ogni triangolo può contribuire con al massimo un nodo, esso seleziona esattamente un letterale vero per clausola, fornendo un'assegnazione di verità coerente e soddisfacente per il 3-SAT.

Problem 4: Max-3SAT

(a) Algoritmo Randomizzato e Complessità 1. Assegnare a ciascuna delle n variabili un valore di verità (True/False) scelto in modo indipendente e casuale con probabilità $\frac{1}{2}$ [cite: 6]. 2. Valutare il numero di clausole soddisfatte da questa assegnazione [cite: 6]. * **Time Complexity:** $O(n + m)$, dove n è il numero di variabili e m il numero di clausole, poiché richiede un tempo lineare per generare l'assegnazione e verificare le clausole [cite: 6].

(b) Dimostrazione del rapporto di approssimazione atteso ($\geq 7/8$) Consideriamo una singola clausola C_j composta da 3 letterali distinti. Poiché l'assegnazione casuale è indipendente, la clausola fallisce se e solo se tutti e tre i suoi letterali risultano falsi contemporaneamente:

$$P(\text{clausola } C_j \text{ fallisce}) = \left(\frac{1}{2}\right)^3 = \frac{1}{8}$$

Di conseguenza, la probabilità che la clausola sia soddisfatta è:

$$P(\text{clausola } C_j \text{ è soddisfatta}) = 1 - \frac{1}{8} = \frac{7}{8}$$

Applicando la linearità del valore atteso, il numero atteso di clausole soddisfatte X è:

$$E[X] = \sum_{j=1}^k E[X_j] = \sum_{j=1}^k \frac{7}{8} = \frac{7}{8}k \geq \frac{7}{8}OPT$$

Questo dimostra che il rapporto di approssimazione atteso è almeno $\frac{7}{8}$ [cite: 6].